

APELLIDOS:

NOMBRE:

(Duración del examen: **1 h 30'**, **10'** para el test, **20'** para las preguntas cortas, **1 hora** para la parte práctica)

PARTE TEÓRICA

Se compone de dos partes: un primer ejercicio que es un test con 6 preguntas y un segundo ejercicio que son dos preguntas de respuesta corta.

Ejercicio 1 (2 Puntos)

Las preguntas acertadas sumarán 1 punto. Las falladas restarán 0.25. Las preguntas no contestadas ni suman ni restan. Hay una única respuesta correcta por pregunta.

- En PLN, el proceso de extraer el significado correcto de una palabra en un contexto determinado es
 - POS Tagging.
 - Word Sense Disambiguation.**
 - Tokenización.
 - Ninguna de las otras respuestas es correcta.
- ¿Cuál de las siguientes afirmaciones sobre Word Embeddings es cierta?
 - CBOW es una arquitectura de embeddings de Word2Vec.
 - Glove se enfoca en un contexto global al crear la matriz de coocurrencia.
 - FastText necesita mayor tiempo de entrenamiento comparado con Word2Vec.
 - Todas las otras respuestas son ciertas.**
- La función de pesado TF-IDF ayuda a
 - Representar las palabras más frecuentes en el documento.
 - Representar las palabras más importantes en el documento.**
 - Representar las palabras menos frecuentes en el documento.
 - Ninguna de las otras respuestas es correcta.
- El recall es una métrica
 - ... que cuantifica el número de predicciones positivas correctas realizadas de todas las predicciones positivas que podrían haberse realizado.**
 - ... que cuantifica el número de predicciones positivas correctas con respecto al número total de predicciones positivas realizadas.
 - ... que cuantifica el porcentaje de aciertos en general.
 - Ninguna de las otras respuestas es correcta.
- Una matriz de coocurrencia es
 - Una matriz documento-término, donde para cada documento se tiene el peso del término del vocabulario en el documento.
 - Una matriz término-término, para medir la coocurrencia de los términos que coaparecen en todos los documentos.
 - Una matriz término-término, para medir la coocurrencia de los términos que coaparecen en el mismo contexto.**
 - Ninguna de las otras respuestas es correcta.

APELLIDOS:

NOMBRE:

Ejercicio 2 (2 Puntos)

- 2.1 [1 Punto] Suponer que se está tratando de clasificar documentos mediante una representación basada en bag-of-words. La entrada es el texto plano en un único String que contiene el texto completo del documento. Describe brevemente cuál sería el pipeline para pasar de la entrada en texto plano hasta un vector de características.

Solución:

Se podría primero tokenizar el texto, preprocesarlo de alguna forma si se quiere hacer una selección de rasgos, de esta forma se obtiene un vocabulario para saber las componentes de los vectores, luego contar frecuencias utilizando alguna función de pesado local o global y con eso ya se podrían construir los vectores, uno para cada documento. La matriz de documento-rasgos podría ser la entrada para un algoritmo de clasificación.

- 2.2. [1 Punto] Dado el siguiente vector para la palabra “watch” (siguiendo un bag-of-words)

[likes, movies, time, escape, football, wrist, prison, night]

¿Cuál sería el vector correcto con pesado binario para las siguientes oraciones y utilizando una ventana de tamaño 10?

- a) “Mary also likes to watch football games.”

Solución:

[1,0,0,0,1,0,0,0]

- b) “John’s new watch shows the time in five locations.”

Solución:

[0,0,1,0,0,0,0,0]

- c) “The investigation clearly shows that he doesn't escape the prison during my watch!”

Solución:

[0,0,0,1,0,0,1,0]

- d) “Duels usually have one scene where the actors go out of frame and you watch their shadows fighting, at least in cliché movies.”

Solución:

[0,1,0,0,0,0,0,0]

APELLIDOS:

NOMBRE:

Cada uno de los vectores binarios tiene una longitud de 8, donde cada posición se corresponde con cada una de las palabras de la lista [likes, movies, time, escape, football, wrist, prison, night]. Para cada oración, se localiza la palabra "watch" y se extraen las diez palabras que la preceden y las diez palabras que la suceden, correspondientes a la ventana de contexto. Por cada palabra de la lista evaluada ([likes, movies, time, escape, football, wrist, prison, night]), se asigna un 1 en la posición del vector binario si aparece al menos una vez dentro de la ventana de contexto o 0 en caso contrario.

PARTE PRÁCTICA

En esta parte se plantean ejercicios cortos relacionados con realizar alguna tarea concreta donde hay que programar en su mayor parte y utilizar librerías vistas en clase.

Nota: La consulta a motores de búsqueda (Google, Bing, etc) y herramientas de inteligencia artificial generativa (ChatGPT, Copilot, Claude, etc) **está terminantemente prohibida**. Sin embargo, puedes acceder a los siguientes enlaces para consultar la documentación pertinente:

- SpaCy: <https://spacy.io/usage/spacy-101>.
- Scikit-learn: <https://scikit-learn.org/stable/api/index.html>
- Numpy: <https://numpy.org/doc/1.26/reference/index.html>

Ejercicio 3 (1 Punto)

Dado el siguiente código de Python:

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Los perros juegan en el parque")

result = []
for token in doc:
    if token.pos_ == "NOMBRE":
        result.append(token.lemma_)

print(result)
```

a) ¿Qué pretende hacer? ¿Crees que es correcto? Justifique la respuesta.

Solución:

APELLIDOS:

NOMBRE:

Se carga un modelo de spaCy y una vez tokenizado (objeto doc tras hacer nlp sobre el texto) se quieren obtener los lemas solo de aquellos tokens cuya categoría gramatical sea un nombre.

El código proporcionado no es correcto por dos motivos:

- El modelo de spaCy es un modelo de inglés, pero el texto está en español, por lo tanto, habría que cargar un modelo en español.
- La etiqueta de Part-of-Speech es incorrecta, no va a reconocer NOMBRE, sino que tiene que ser NOUN para que funcione.

b) Si consideras necesario realizar algún cambio, haz una propuesta de código alternativo.

Solución:

- Se debe de cambiar el modelo de SpaCy "en_core_web_sm" por "es_core_news_sm".
- Se debe de poner la etiqueta PoS de spaCy correcta ("NOUN" en vez de "NOMBRE").

```
import spacy

nlp = spacy.load("es_core_news_sm")
doc = nlp("Los perros juegan en el parque")

result = []
for token in doc:
    if token.pos_ == "NOUN":
        result.append(token.lemma_)

print(result)
```

Ejercicio 4 (2 Puntos)

Tras realizar un extensivo preprocesamiento sobre un conjunto de textos, se han obtenido las siguientes ventanas de contexto de tamaño igual a tres:

```
windows = [
    ["gato", "duerme", "sofa"],
    ["gato", "salta", "cama"],
    ["gato", "mira", "parque"],
    ["perro", "juega", "sofa"],
    ["perro", "duerme", "cama"],
    ["perro", "juega", "parque"]
]
```

APELLIDOS:

NOMBRE:

- a) [1.5 Puntos] Para las ventanas de contexto anteriores, obtén su correspondiente matriz de coocurrencias y muéstrala utilizando DataFrame.

Solución:

Un código básico que podría resolver este apartado es el propuesto a continuación:

```
import numpy as np
import pandas as pd

vocab = sorted(set(word for window in windows for word in window))
vocab_idx = {word: i for i, word in enumerate(vocab)}

cooccurrence_matrix = np.zeros((len(vocab), len(vocab)))
for win in windows:
    for word_i in win:
        for word_j in win:
            if word_i != word_j:
                cooccurrence_matrix[vocab_idx[word_i], vocab_idx[word_j]] += 1

pd.DataFrame(cooccurrence_matrix, index=vocab, columns=vocab, dtype=int)
```

	cama	duerme	gato	juega	mira	parque	perro	salta	sofa
cama	0	1	1	0	0	0	1	1	0
duerme	1	0	1	0	0	0	1	0	1
gato	1	1	0	0	1	1	0	1	1
juega	0	0	0	0	0	1	2	0	1
mira	0	0	1	0	0	1	0	0	0
parque	0	0	1	1	1	0	1	0	0
perro	1	1	0	2	0	1	0	0	1
salta	1	0	1	0	0	0	0	0	0
sofa	0	1	1	1	0	0	1	0	0

Primero se construye el vocabulario de todas las palabras que aparecen en las ventanas de contexto, ordenándolas de forma alfanumérica (con el fin de facilitar posteriormente la visualización). En segundo lugar, se crea un diccionario que asigna a cada palabra un índice numérico.

APELLIDOS:

NOMBRE:

A continuación, se inicializa a cero una matriz cuadrada donde tanto las filas como las columnas son las palabras del vocabulario construido. Iterando sobre las palabras de las ventanas de contexto, por cada par de palabras distintas, se incrementa en 1 la correspondiente celda en la matriz creada, usando para ello el diccionario construido anteriormente.

Por último, para facilitar su visualización se convierte la matriz construida en un DataFrame de Pandas. Cabe destacar que, al ser la matriz simétrica, este código podría hacerse de manera mucho más eficiente.

- b) [0.5 Puntos] Usando la matriz de coocurrencias calculada, obtén el grado de similitud entre las palabras "perro" y "gato".

Solución:

Dada la matriz de coocurrencias calculada, obtenemos los vectores de coocurrencia para las palabras perro y gato. Tras ello, una posible solución para calcular su grado de similitud es calcular la similitud coseno de estos dos vectores.

```
from sklearn.metrics.pairwise import cosine_similarity

perro_vector = cooccurrence_matrix[vocab_idx['perro']]
gato_vector = cooccurrence_matrix[vocab_idx['gato']]

print(cosine_similarity([perro_vector], [gato_vector]))

# Output: [[0.57735027]]
```

Ejercicio 5 (3 Puntos)

Dada la frase de consulta "Dogs run fast" y el conjunto de frases a comparar

```
["Canines run quickly", "Dogs are pets", "Sharks move fast"]
```

- a) [1 Punto] Crea la matriz de pesado binario de las frases anteriores (tanto la frase de consulta como el conjunto de frases a comparar).

Solución:

Podemos construir la matriz de pesado binario pasándole tanto la frase de consulta como las frases a comparar al método de Scikit-learn CountVectorizer(binary=True).

APELLIDOS:

NOMBRE:

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
query = "Dogs run fast"
sentences = [
    "Canines run quickly",
    "Dogs are pets",
    "Sharks move fast"
]
```

```
vectorizer = CountVectorizer(binary=True)
X = vectorizer.fit_transform([query] + sentences)
```

- b) [0.5 Puntos] Empleando la anterior matriz de pesado binario, calcula el grado de similitud entre la frase de consulta y cada una de las frases del conjunto de frases a comparar.

Solución:

Dada la matriz binaria anteriormente calculada, cogemos las vectorizaciones binarias para la frase de consulta (primera fila de la matriz) y para las frases a comparar (resto de filas de la matriz). Tras ello, una posible solución para calcular su grado de similitud es calcular la similitud coseno de estos dos vectores.

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
query_vector = X[0]
sentences_vectors = X[1:]

print(cosine_similarity(query_vector, sentences_vectors))

# Output: [[0.33333333 0.33333333 0.33333333]]
```

- c) [1 Punto] Utiliza embeddings a nivel de palabra para calcular nuevamente el grado de similitud entre la frase de consulta y el conjunto de frases. Para ello, carga el modelo de Word2Vec "word2vec-google-news-300" mediante el siguiente código de ejemplo:

```
import gensim.downloader as api
import numpy as np

w2v = api.load("word2vec-google-news-300")
```

Solución:

APELLIDOS:

NOMBRE:

Tanto para la frase de consulta como para las frases a comparar, primero se debe realizar un proceso de tokenización. A continuación, por cada token obtenido, se debe calcular su correspondiente embedding con el modelo de Word2Vec cargado. Una vez obtenidos los embeddings de los tokens que componen una frase, una posible solución para obtener una representación de cada frase es hacer la media de los embeddings de los tokens que lo componen.

Tras calcular los embeddings a nivel de frase, nuevamente una posible solución para calcular el grado de similitud entre la frase de consulta y las frases a comparar es calcular su similitud coseno.

```
emb_query = np.mean([w2v[token.lower()] for token in query.split()], axis=0)
```

```
for sent in sentences:
```

```
    emb_sent = np.mean([w2v[token.lower()] for token in sent.split()], axis=0)  
    print(cosine_similarity([emb_query], [emb_sent]))
```

```
# Output: [[0.80121315]] [[0.65012157]] [[0.5505907]]
```

- d) [0.5 Puntos] Compara y analiza las diferencias encontradas entre los valores de similitud obtenidos a través de la matriz de pesado binario y los embeddings a nivel de palabra. Explica si los valores obtenidos son razonables.

Solución:

Cuando se utiliza la matriz documento-rasgos utilizando bolsa de palabras, no se tiene en cuenta el orden de estas y se asume su independencia en la representación. Esto lleva a que esté devolviendo exactamente el mismo valor de similitud entre la consulta y las otras tres frases, solo porque en cada una de ellas se comparte un rasgo con la consulta. Sin embargo, hay una que es claramente más similar que las otras dos con la consulta, que sería la primera, y eso es lo que dice precisamente el cálculo de la similitud cuando se representa con embeddings.

Por todo ello se puede decir lo siguiente:

- Al usar la matriz de pesado binario, se obtiene el mismo valor de similitud en las tres frases a comparar ([0.33333333 0.33333333 0.33333333]), mientras que con Word2Vec no (con el código propuesto, se obtiene [0.80121315, 0.65012157, 0.5505907]).
- El uso de embeddings de Word2Vec da valores de similitud mucho más coherentes. Por ejemplo, se obtiene que la primera frase a comparar es más similar a la frase de consulta que el resto de las frases.

APELLIDOS:

NOMBRE:

- Esto se debe a que la matriz de pesado binario está teniendo en cuenta únicamente coincidencias de términos y, por tanto, los valores de similitud se deben a que en cada una de las frases a comparar solo hay una coincidencia con la frase de consulta.
- Por el contrario, en Word2Vec términos semánticamente similares tienden a tener embeddings similares, permitiendo una mejor comparación incluso cuando los términos exactos difieren.